2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
October 23-27, 2022, Kyoto, Japan

Advanced Skills by Learning Locomotion and Local Navigation End-to-End

Nikita Rudin[1,2], David Hoeller[1,2], Marko Bjelonic[1] and Marco Hutter[1]

***Abstract*— The common approach for local navigation on challenging environments with legged robots requires path planning, path following and locomotion, which usually requires a locomotion control policy that accurately tracks a commanded velocity. However, by breaking down the navigation problem into these sub-tasks, we limit the robot's capabilities since the individual tasks do not consider the full solution space. In this work, we propose to solve the complete problem by training an end-to-end policy with deep reinforcement learning. Instead of continuously tracking a precomputed path, the robot needs to reach a target position within a provided time. The task's success is only evaluated at the end of an episode, meaning that the policy does not need to reach the target as fast as possible. It is free to select its path and the locomotion gait. Training a policy in this way opens up a larger set of possible solutions, which allows the robot to learn more complex behaviors. We compare our approach to velocity tracking and additionally show that the time dependence of the task reward is critical to successfully learn these new behaviors. Finally, we demonstrate the successful deployment of policies on a real quadrupedal robot. The robot is able to cross challenging terrains, which were not possible previously, while using a more energy-efficient gait and achieving a higher success rate. Supplementary videos can be found on the project website: `https://sites.google.com/leggedrobotics.com/end-to-end-loco-navigation`**

I. INTRODUCTION

Legged robots have the potential to navigate challenging environments, such as in Fig. 1, much better than wheeled robots, but they require complex motions to do so. Multiple control methods have been proposed including Model Predictive Control [1], [2], trajectory optimization [3], [4], imitation learning [5], [6], and fully learned approaches [7], [8]. Recently, deep reinforcement learning has proven to be a powerful tool to achieve robust locomotion. It has brought the robots out of a controlled lab environment and allowed them to evolve in the real world, using perceptive and proprioceptive information to walk on obstacles such as stairs, rocks, or roots [9].

To achieve a robust execution in the real world, the controller must be capable of dynamic motions while handling model mismatches, noisy sensors, reacting to external disturbances, missteps, and slippage. Nonetheless, these policies still fail to exhibit the level of agility and precision of animals or humans. In all of these cases, the control policy only needs to slightly modulate a periodic motion of the joints. In contrast, animals can switch gaits, plan a series of consecutive steps, perform jumps, and use contacts on other parts of their body when necessary. If we want our policies to achieve the same level of dexterity, they need to explore the entire solution space by removing assumptions and implicit constraints that limit the possible behaviors.



Fig. 1: The legged robot ANYmal navigating around obstacles, climbing stairs, and jumping over a gap.

A. Related Work

Most of the related work splits the local navigation task of legged robots into multiple sub-parts. For example, a planing module generates a trajectory that the robot can follow to reach its target. Then a path-following controller steers the robot towards that path by providing velocity commands to the locomotion controller, which tries to match those commands by moving the legs of the robot [10], [11]. Similarly, when operators manually guide the robot through a local environment with a joystick, they send velocity commands to the locomotion controller. As such, velocity tracking is commonly used as the task for both model-based [1], [2], [12] and learned locomotion controllers [7], [8], [9], [13].

This is a natural way to split the locomotion and navigation problems. Unfortunately, it imposes unintended restrictions on the behavior of the low-level policy and thus limits the possibilities of the overall system. For example, when jumping over a gap, the robot needs to be free to modulate its velocity before and after the jump. Furthermore, the locomotion policy must be allowed to slow down or even stop in dangerous situations. These cases would result in higher velocity tracking costs making the desired behaviors harder to achieve. Finally, keeping the base at a constant velocity encourages the policy to learn a gait, which is not necessarily the most energy-efficient. In the case of four-legged locomotion, this results in the trotting gait seen in all learning approaches mentioned above. Even a policy trained

[1] Robotic Systems Lab, ETH Zurich. [2] NVIDIA, contact: rudinn@ethz.ch

2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) | 978-1-6654-7927-1/22/$31.00 ©2022 IEEE | DOI: 10.1109/IROS47612.2022.9981198

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 02,2026 at 15:49:53 UTC from IEEE Xplore. Restrictions apply.

specifically to generate gaits, only learns to output different versions of trotting [14].

Trajectory optimization methods can directly produce joint commands based only on a target state and a cost to minimize [3], [15]. Unfortunately, these methods are computationally expensive and lack the robustness of reinforcement learning policies. A simple position tracking task would penalize the distance between the robot and the target location continuously. However, the optimal solution would be to reach the goal as fast as possible, resulting in a behavior close to the "bang-bang" solution of optimal control problems [16]. This is not the desired behavior for a real-world robot as it would result in needlessly aggressive and dangerous motions. This problem can be mitigated by penalizing velocities and torques, but the final behavior depends on the relative scale of these penalties, which must be tuned individually for each case. Additionally, this set-up also penalizes trajectories where the robot slows down or deviates from the shortest path.

Previous works have explored the agile locomotion of legged robots. Trajectory optimization has been shown to produce agile motions, including jumping and climbing behavior [15]. However, this approach requires costly optimizations for each trajectory, assumes the model to be continuously differentiable, and the authors only show hardware experiments on simple cases. Learning approaches, splitting the locomotion problem into a foothold planner and a tracker [17], [18], achieved locomotion over challenging terrains, including gaps and stepping stones. Despite impressive results, this approach requires a multi-stage training approach, assumes contact patterns that would fail on our climbing task, and was again not shown to transfer to the real world. For selected tasks, multiple approaches resulted in dynamic and agile motions on real robots. These include: jumping over obstacles with a model-based planer [19], jumping onto a table with trajectory optimization [20], exhibiting cat-like landing capabilities with a model-based approach [21], a learned policy [22], as well as a combination of both [23].

Learning approaches have been applied to local navigation of a legged robot. The problem can be split into a high-level policy steering the low-level locomotion controller [24], or combined into a single end-to-end policy [25]. However, in both cases the locomotion controllers are trained using velocity commands resulting in limited capabilities of the system.

B. Contribution

In this work, we propose a position-based formulation where the robot must reach the target location only after a defined time. The task reward is based on the final distance to the target at the end of an episode and is not influenced by the trajectory the robot took to get there. We need to add multiple penalties to achieve sim-to-real transfer, but this setup has the advantage of decoupling the main task from these costs. As long as the robot reaches the target position, it is free to optimize its path and gait to minimize the additional penalties without influencing the main task reward. This creates a hierarchical reward structure where the policy first learns to solve the task before optimizing the quality of motion needed to do so.

Using the velocity tracking and continuous position approaches as baselines for our experiments, we show that our method allows the policy to succeeds on common terrains such as stairs and slopes with increased difficulty. Furthermore, it enables the training complex behaviors such as jumping over gaps, climbing on boxes, and avoiding obstacles. Additionally, the policy learns a novel locomotion gait resulting in lower failure rates and energy consumption. Finally, our approach removes the need for a local planner since the locomotion policy directly learns this capability. The policies are deployed on the real robot and run in real-time on the onboard computer.

II. METHOD

We train a neural network policy to solve various locomotion tasks for a quadrupedal robot. The policy receives measurements provided by the robot's sensors, and outputs commands that are directly sent to the motors. The neural network is trained in simulation using Deep RL, after which it is transferred to the real robot without any further training or adaptation. We build upon the massively parallel set-up of [26], with Isaac Gym [27] simulation and Proximal Policy Optimization (PPO) [28] as the Deep RL algorithm.

A. Observation and Action Space

The observations include joint positions, joint velocities, base linear and angular velocity, commands, previous actions, and terrain measurements sampled around the robot. The commands are defined as the three-dimensional location of the target expressed in the base frame and the remaining time to reach that location. Noise is added to the observations based on the noise level of the robot's sensors. The actions are interpreted as target joint positions given to the motors where a PD controller transforms them into joint torques.

B. Rewards

The rewards are split into two parts: task rewards only active at the end of the episode, and various penalties provided throughout the episode.

1) Task Reward: The task reward instructs the robot to reach the provided target location. It is calculated as

$$r_{task} = \begin{cases} \frac{1}{T_r} \frac{1}{1+\|\mathbf{x}_b - \mathbf{x}_b^*\|^2}, & \text{if } t > T - T_r \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

where $\mathbf{x}_b$ and $\mathbf{x}_b^*$ are the current and target positions of the base, t is the time since the beginning of the episode, T is the maximum episode length and T_r is a parameter defining the duration of the reward. T_r needs to be long enough to force the policy to stop at the target in a stable configuration. Otherwise, the policy learns to jump or lean towards the target at the last moment, which would result in a crash on the real robot where there are no episode resets.

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 02,2026 at 15:49:53 UTC from IEEE Xplore. Restrictions apply.

2) Penalties: We penalize joint accelerations, joint torques, collisions, and abrupt actions changes. Additionally, we introduce a new reward penalizing feet accelerations. It reduces impacts with the ground, causing loud noises and potential damage to the motors and structure. The complete set of penalties is computed as

$$\begin{aligned} r_{penalties} = &- c_1\|\ddot{\mathbf{q}}\|^2 - c_2\|\boldsymbol{\tau}\|^2 - c_3 N_c \\ &- c_4\|\mathbf{a} - \mathbf{a}_{-1}\|^2 - c_5 \sum_{feet} \|\ddot{\mathbf{x}}_f\|^2 \end{aligned} \tag{2}$$

where $\mathbf{q}$, $\boldsymbol{\tau}$, N_c $\mathbf{a}$, and $\mathbf{x}_f$ represent joint positions, joint torques, number of collisions, actions, and positions of the feet respectively, and c_{1-5} are scaling constants.

There is an important difference in how these penalties interact with the main task. With velocity tracking, the policy must compromise between minimizing these penalties and optimizing the main task. On the other hand, minimizing these penalties does not counteract the main reward with our approach. As long as the robot reaches the target in time, it can freely minimize the different quantities without negatively affecting the position tracking. This simplifies the tuning process and removes the need to compromise on the quality of the solution.

3) Exploration Reward: Due to the sparse nature of the main reward, we find it beneficial to add a reward term encouraging exploration at the beginning of training. In order to bias the policy to walk towards the target, we reward it for any base velocity in the correct direction. The reward is defined as

$$r_{bias} = \frac{\dot{\mathbf{x}}_b \cdot (\mathbf{x}_b^* - \mathbf{x}_b)}{\|\dot{\mathbf{x}}_b\| \|\mathbf{x}_b^* - \mathbf{x_b}\|} \tag{3}$$

This rewards is automatically removed after a few training iterations, and thus doesn't constrain the final solution learned by the policy. r_{bias} is removed once r_{task} reaches $50\,\%$ of it's maximum value.

4) Stalling Penalty: PPO optimizes the discounted sum of rewards over the episode, with a discount factor that must be lower than 1 for stability. This has an unintended effect on the optimal solution to the problem. The policy receives two types of rewards, a positive reward at the end of an episode representing the main task and negative penalties throughout the episode, which are larger when the robot is walking. In order to optimize the discounted sum of rewards, it is beneficial to push the negative penalties as far into the future as possible. Therefore, the policy learns to wait until the last moment when it suddenly runs fast towards the target. In practice, we would like the robot to either use all of the provided time or wait at the target. Furthermore, the real-world energy consumption should be optimized without a discount factor. In order to counterbalance this effect, we add a small penalty waiting while being far away from the target. It is defined as

$$r_{stall} = \begin{cases} -1, & \text{if} \quad \|\dot{\mathbf{x}}_b\| < 0.1\,\mathrm{m/s} \\ & \text{and } \|\mathbf{x}_b - \mathbf{x}_b^*\| > 0.5\,\mathrm{m} \\ 0, & \text{otherwise.} \end{cases} \tag{4}$$

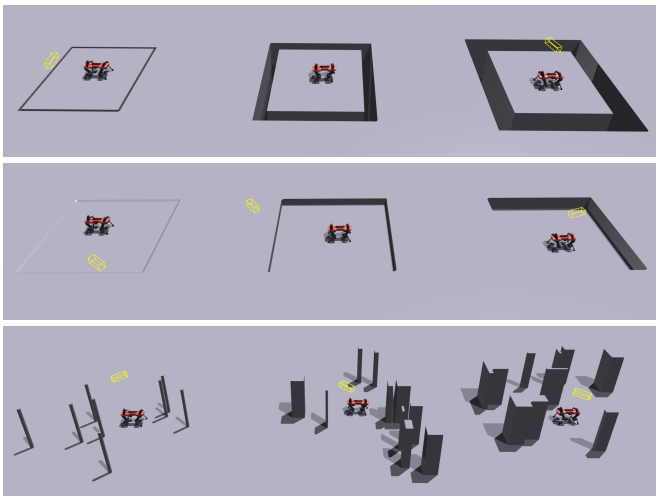


Fig. 2: Examples of three newly added terrains types with a representative curriculum from the easiest version on the left to the hardest terrain seen during training on the right.

C. Terrains

For a proper comparison with velocity tracking, we first train a policy on terrains from [26]. These include slopes, stairs, random rectangular obstacles, and rough surfaces. We then explore terrains requiring more agile and complex maneuvers. We evaluate our approach on three new tasks represented in Fig. 2. First we create a terrain where the policy needs to step or jump over a gap. The size of the gap is such that keeping a constant base velocity is too constraining since the robot needs to leap forward to cross the distance and inevitably slows down after the jump side since it barely reaches the other side. Second, we explore the use of other parts of the body by creating a climbing task. In this scenario the robot must climb out of a pit, which is too deep to simply step out of it. The policy needs to learn a complex series of actions where it first puts the front feet up before using the back knees to help it reach the top. Finally, we test the local planning capabilities of our policy by creating a terrain with high obstacles which the policy must avoid. When an obstacle is in its way, the policy must learn to disregard the direct path towards the target and find a way around the obstacle.

D. Episode Process

At the beginning of each episode, a valid position target is sampled for each robot. The sampling is done uniformly in polar coordinates around the initial position of the robot with a radius between $1\,\mathrm{m}$ and $5\,\mathrm{m}$. The height of the target is set to $0.5\,\mathrm{m}$ above the ground. A target is considered invalid if it overlaps with holes or high obstacles. Invalid targets are re-sampled until none are left. The episode length is set to $T = 6\,\mathrm{s}$, with the task reward provided after $5\,\mathrm{s}$ for a duration of $T_r = 1\,\mathrm{s}$. At the end of the episode, the progress of each robot is evaluated to update the terrain curriculum. A terrain is considered successfully solved if the robot finishes the episode within $0.5\,\mathrm{m}$ of the target.

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 02,2026 at 15:49:53 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Maximum difficulty of terrains solved with a $95\,\%$ success rate. V: velocity tracking, P_c: continuous position tracking, P_f: final position tracking (ours).

Terrain type	Parameter	95 % Success		
		V	P_c	P_f (ours)
stairs (1)	rise	$0.22\,\mathrm{m}$	$0.2\,\mathrm{m}$	$0.4\,\mathrm{m}$
slope (2)	angle	$35\,\mathrm{deg}$	$21\,\mathrm{deg}$	$48\,\mathrm{deg}$
random steps (3)	height	$\pm 0.14\,\mathrm{m}$	$\pm 0.16\,\mathrm{m}$	$\pm 0.2\,\mathrm{m}$
obstacles (4)	length	$0.35\,\mathrm{m}$	-	$0.85\,\mathrm{m}$
gap (5)	width	$0.15\,\mathrm{m}$	$0.65\,\mathrm{m}$	$1.2\,\mathrm{m}$
pit (6)	depth	$0.1\,\mathrm{m}$	$0.65\,\mathrm{m}$	$0.95\,\mathrm{m}$

E. Training Stability

We have found that while our approach can lead to better-performing policies, it is also more sensitive to training instabilities. In particular, predicting the *value function* seems to be more difficult with our temporally sparse task reward. The fact that the task reward is only provided at the end of the episode and only if the robot does not crash until then makes it harder for the *critic* to predict the correct outcome. This makes the whole system prone to instabilities, where a random seed change can cause a failed training run. We use the PPO implementation from [26] with similar hyper-parameters. However, we introduce some modifications which have proven effective to mitigate the instabilities described above.

First, we increase the batch size by doubling the number of consecutive steps each robot takes per policy iteration. We use 4096 parallel robots performing 48 steps resulting in a *batch size* of $200'000$ samples. Additionally, we reduce the episode length from $20\,\mathrm{s}$ to $6\,\mathrm{s}$. Finally, the *value function bootstrapping* was introduced to mimic an infinite horizon problem, which is necessary in the case of velocity tracking since there is no termination condition for this task. However, in our case, the time until task termination is clearly defined and provided as input to both the *actor* and the *critic*. We, therefore, remove the bootstrapping and obtain more stable training. We train all policies for a total of 2000 iterations requiring approximately 1 hour of training.

III. Simulation Experiments

In this section, we compare our approach to the velocity tracking task and a continuous position tracking reward. The velocity tracking policy is trained as in [26] with the only difference being reward scales and command ranges. We use the penalties described in Sec. II-B.2 with the same weights while scaling the task rewards to a magnitude similar to the tracking reward of the new approach. In order to simplify the task, we remove sideways velocity commands keeping only forward/backward and yaw. Continuous position tracking is achieved by providing r_{task} at every time-step ($T_r = T$) without any further tuning.

A. Maximum Terrain Difficulty

We compare the capabilities of the three approaches by evaluating the maximum difficulty of terrains each policy can solve. We train policies on a single terrain type with an increased number of terrain levels and higher maximum complexity. After training for the same number of iterations, we evaluate the policies on each terrain difficulty and find the highest level they are able to solve. The robots must cross $4\,\mathrm{m}$ of the terrain within $8\,\mathrm{s}$. The velocity tracking policy receives a constant forward velocity command of $1\,\mathrm{m/s}$, and we consider a terrain solved if the robot has walked more than $4\,\mathrm{m}$ even if it crashed afterwards. For our approach, the policy receives a target $5\,\mathrm{m}$ in front of its initial position, and we consider an episode successful if the final position of the robot is within $0.5\,\mathrm{m}$ of the target and no crash has occurred. Even with a setup favoring velocity tracking, our approach outperforms both methods. In Table I, we report the highest terrain difficulty on which the different policies have a success rate of at least $95\,\%$. The corresponding terrains are shown in Fig. 3

Our approach raises the achieved difficulty on all terrains compared to velocity tracking. On stairs and slopes, it increases the step height and slope angle by $80\,\%$ and $35\,\%$, which pushes the capabilities of the robot beyond what it would encounter in standard real-world conditions. Random rectangular obstacles remain a challenge due to the poor configuration of some of the generated terrains. Nevertheless, our approach increases the maximum obstacle height by $40\,\%$. Finally, the most significant difference appears in our newly added terrains. The velocity tracking policy can only solve trivial terrain difficulties requiring minor adjustments to a standard walking gait. Our approach helps the policy learn more complex behaviors allowing the robot to perform jumps, climb using its knees and avoid obstacles of non-trivial size. Quantitatively, the size of obstacles, length of gaps, and depth of pits are increased by $140\,\%$, $700\,\%$, and $850\,\%$, respectively.

On most terrains, the continuous position tracking approach achieves higher difficulties than velocity tracking. However, it completely fails to navigate obstacles with a sufficient reliability, and is inferior to final position tracking on all terrains.

B. Energy Consumption

The policy does not need to compromise between maximizing task rewards and minimizing penalties with our approach. Furthermore, the robot's gait is less restricted than with the velocity tracking approach. In order to verify this hypothesis, we evaluate the energy consumption of policies trained with various approaches. We use the sum of squared torques as a proxy for energy since it is the penalized quantity during training and is proportional to heat losses in the motors.

We record the total energy consumption while the robots walk across $100\,\mathrm{m}$ of flat terrain at various velocities. While it is straightforward to command the velocity tracking policy, our approach requires some modification to its commands. We can achieve a pseudo velocity control by fixing the target to be always $3\,\mathrm{m}$ in front of the robot and setting a constant value instead of the time left to reach that target. By providing a fixed target and time, the policy

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 02,2026 at 15:49:53 UTC from IEEE Xplore. Restrictions apply.

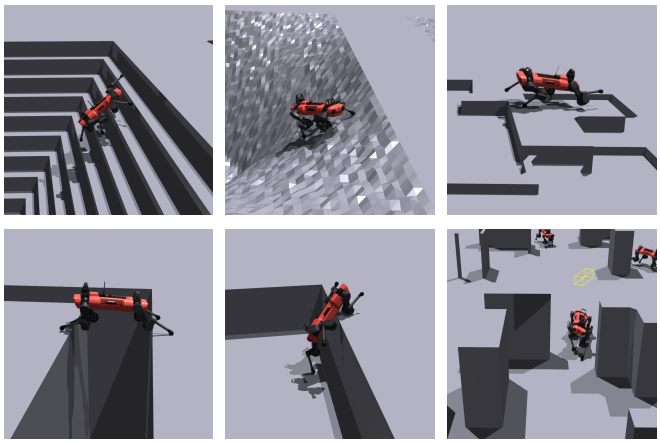


Fig. 3: Simulation deployment of policies for each terrain type. The terrains represent the highest difficulty successfully solved by our approach with a 95 % success rate. Parameters of these terrains are provided in Table I.

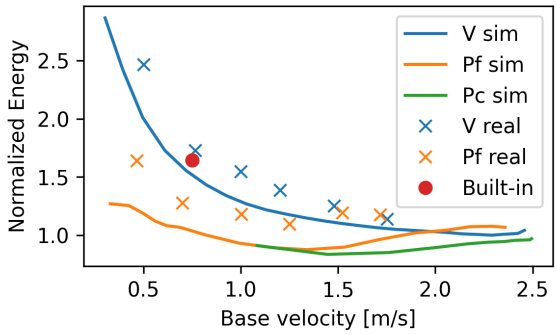
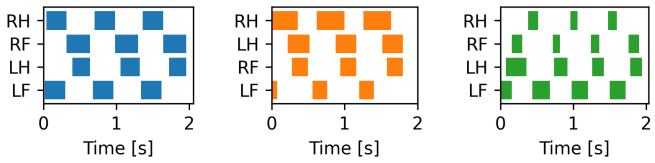


Fig. 4: Normalized cost of transport of three approaches in simulation and on hardware. The sum of squared torques is recorded while the policies receive a constant command over 100 m in simulation and 5 m on the real robot. V: velocity tracking, P_c: continuous position tracking, P_f: final position tracking (ours). Built-in: learning based controller provided by the manufacturer.

can walk continuously even though it was never trained in that scenario. Furthermore, varying this fixed time directly influences the velocity of the robot. When the policy believes it has a lot of time to reach the target, it will adopt a slow and safe gait. As that time is reduced, it will try to close the gap faster and faster. We also modify the commands of the continuous position tracking policy. This time we change the distance to the target since the policy is not conditioned on time.

In Fig. 4 we report the sum of squared torque per traveled distance for all approaches at different velocities. Interestingly, all approaches converge to similar values at high velocities, but the velocity tracking policy performs worse at slow speed. In that regime, keeping the base at constant velocity seems to have a stronger impact on the energy consumption. Both position based approaches present similar results, with the continuous tracking outperforming



(a) V (b) P_f (c) P_c

Fig. 5: Emerging gait patterns of three task definitions. (a): Velocity tracking, trotting gait. (b): Final position tracking (ours), three-phased gait towards the front-left side. (c): Continuous position tracking, three-phased gait towards the right-hind side.

our approach at higher velocities. We note that this difference is expected since the continuous tracking policy is trained to run as fast as possible, while our approach is mostly trained with velocities around 1 m/s (5 m in 5 s).

We see higher torques in hardware experiments compared to our simulated results. The offset can explained by the restrictive limits we impose on our actuators in simulation. The real robot is able to produce more torque than what the policy experienced in simulation. Nevertheless, we see similar trend, with our approach outperforming the velocity tracking policy at low speed and converging to similar torques at higher velocities. We do not transfer the continuous position tracking policy, since it is prone to aggressive motions which could potentially damage the robot.

C. Emergent Gaits

Most previous works on learned locomotion resulted in a trotting gait. This can be explained by the fact that the robot's base can keep a nearly constant velocity. As such, velocity tracking approaches are heavily biased towards trotting. However, walking with a constant base velocity is not common for animals as it requires additional effort. With our approach, we remove these artificial constraints, which allow the emergence of different types of gaits. Furthermore, for velocity tracking policies, it is necessary to encourage larger steps by rewarding the air time or clearance of the feet [29], [26]. This is needed because the policy prefers to do many small steps in order to follow the command with minimal oscillations of the base. With our approach, this reward is no longer necessary since base oscillations are no longer penalized.

Fig. 5 compares the gaits of velocity and position tracking. We see that the policy learns a three-phased asymmetric gait with our approach. Interestingly, the robot moves diagonally, probably due to the kinematics and motor arrangement, giving the most extended reach and maximum power in this direction. In Fig. 5, the robot is moving towards the front left. The gait is split into three phases. First, the back (right hind) leg touches the ground, followed by the center (left hind and right front) legs. Finally, the forward facing (left front) leg is briefly in contact. The continuous tracking policy uses a similar gait, but it learns to walk towards it's right back side, meaning that the contact schedule is inverted.

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 02,2026 at 15:49:53 UTC from IEEE Xplore. Restrictions apply.



Fig. 6: Deployment of policies on the real robot. Top Left: running on flat ground, top right: climbing stairs, bottom left: jumping over a $0.6\,\mathrm{m}$ gap, bottom right: navigating around obstacles.

Qualitatively, as can be seen in the accompanying video, the robot's motion is more dynamic and is generally perceived to be more natural with our approach. The combination of a stronger base motion and large smoother steps gives the robot an organic appearance, which the velocity tracking policy lacks.

However, there is an unfortunate artifact in the behavior. The policy only learns to walk in one direction, preferring to turn instead of walking sideways or backward. This can become a problem when we want the robot to walk back and forth repetitively. This seems to be due to a very attractive local minimum as every training run results in a random but fixed walking direction.

D. Hardware Deployment

We deploy policies trained with our approach on the real robot. Following previous work [29], we augment the simulation with a neural network trained to mimic the series elastic actuators of the robot. Unfortunately, the actuator network is not accurate enough in the high-velocity regime. At maximum velocity, it still allows high torques, which is not possible for a brushless DC motor [30]. We model the maximum torque as an affine function of motor velocity and clip the torques required by the policy to those maximum values.

During deployment, commands can be specified in two ways: either with real position targets or using a joystick with the pseudo velocity control described in Sec. III-B. In the second case, the direction of the joystick command defines the target direction, while the magnitude is used to scale the remaining time given to the policy. This proves to be a natural way to command the robot, and the policy responds well to quickly changing commands even though it was not trained for it.

For practical reasons, we do not use the most complicated terrains described in Table I. Nevertheless, we show that our approach allows the robot to perform maneuvers that were previously not possible. We show our policies successfully running on flat terrain, climbing stairs at high speed, jumping over a virtual $0.6\,\mathrm{m}$ gap, and navigating around obstacles. These motions are represented in Fig. 6 as well as in the supplementary video.

The success rate of deployed policies greatly depends on the terrain type. While it is similar to I on stairs, it is lower for harder terrains. This is due to our reliance on provided state estimation and perception modules which were not designed for jumping or climbing.

IV. Discussion

In this work, we have proposed to rethink the task definition used to train locomotion policies for legged robots. We have shown that specifying a target location instead of a base velocity command reduces the constraints on the solution space. This set-up decouples the task from the optimization of other quantities, lets the policy learn a more energy-efficient gait, and most importantly, helps it learn more complex maneuvers. On the downside, our approach seems more prone to training instabilities. Another challenge is that the robot only learns to walk in one direction and does not use its kinematic symmetry. The task should also be extended to allow the specification of multiple consecutive way-points without discontinuities in the motion. Finally, the reliance on state estimation and perception has to be reduced or these modules need to be improved for our specific use cases. These challenges should be investigated as part of future work.

Our approach can be applied to a large set of tasks and robots. It is not limited to locomotion on challenging terrain. For example in a fall recovery scenario, the reward encouraging the robot to stand up can also be activated only at the end of an episode, letting the policy free to choose the trajectory instead of requiring it to recover as fast as possible.

By showing that these relatively small changes in the training process lead to great improvements in the agility of the robot, we emphasizes that we are still far from exploiting the full capabilities of our robots, and encourage other researchers to apply similar techniques to their respective tasks.

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 02,2026 at 15:49:53 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] D. Kim, J. D. Carlo, B. Katz, G. Bledt, and S. Kim, "Highly dynamic quadruped locomotion via whole-body impulse control and model predictive control," *ArXiv*, vol. abs/1909.06586, 2019.

[2] M. Neunert, M. Stäuble, M. Giftthaler, C. D. Bellicoso, J. Carius, C. Gehring, M. Hutter, and J. Buchli, "Whole-body nonlinear model predictive control through contacts for quadrupeds," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1458–1465, 2018.

[3] T. Apgar, P. Clary, K. Green, A. Fern, and J. W. Hurst, "Fast online trajectory optimization for the bipedal robot cassie." in *Robotics: Science and Systems*, vol. 101, 2018, p. 14.

[4] M. Bjelonic, P. K. Sankar, C. D. Bellicoso, H. Vallery, and M. Hutter, "Rolling in the deep – hybrid locomotion for wheeled-legged robots using online trajectory optimization," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3626–3633, 2020.

[5] X. B. Peng, E. Coumans, T. Zhang, T.-W. E. Lee, J. Tan, and S. Levine, "Learning agile robotic locomotion skills by imitating animals," in *Robotics: Science and Systems*, 07 2020.

[6] A. Reske, J. Carius, Y. Ma, F. Farshidian, and M. Hutter, "Imitation learning from mpc for quadrupedal multi-gait control," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 5014–5020.

[7] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[8] J. Siekmann, K. Green, J. Warila, A. Fern, and J. W. Hurst, "Blind bipedal stair traversal via sim-to-real reinforcement learning," *CoRR*, vol. abs/2105.08328, 2021. [Online]. Available: https://arxiv.org/abs/2105.08328

[9] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Science Robotics*, vol. 7, no. 62, p. eabk2822, 2022.

[10] D. Wooden, M. Malchano, K. Blankespoor, A. Howardy, A. A. Rizzi, and M. Raibert, "Autonomous navigation for bigdog," in *2010 IEEE International Conference on Robotics and Automation*, 2010, pp. 4736–4741.

[11] C. D. Bellicoso, M. Bjelonic, L. Wellhausen, K. Holtmann, F. Günther, M. Tranzatto, P. Fankhauser, and M. Hutter, "Advances in real-world applications for legged robots," *Journal of Field Robotics*, vol. 35, no. 8, pp. 1311–1326, 2018. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/rob.21839

[12] Y. Guo, M. Zhang, H. Dong, and M. Zhao, "Fast online planning for bipedal locomotion via centroidal model predictive gait synthesis," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 6450–6457, 2021.

[13] G. Ji, J. Mun, H. Kim, and J. Hwangbo, "Concurrent training of a control policy and a state estimator for dynamic and robust legged locomotion," 2022.

[14] Y. Yang, T. Zhang, E. Coumans, J. Tan, and B. Boots, "Fast and efficient locomotion via learned gait transitions," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Faust, D. Hsu, and G. Neumann, Eds., vol. 164. PMLR, 08–11 Nov 2022, pp. 773–783.

[15] A. W. Winkler, C. D. Bellicoso, M. Hutter, and J. Buchli, "Gait and trajectory optimization for legged systems through phase-based end-effector parameterization," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1560–1567, 2018.

[16] R. Bellman, I. Glicksberg, and O. Gross, "On the "bang-bang" control problem," *Quarterly of Applied Mathematics*, vol. 14, no. 1, pp. 11–18, 1956.

[17] V. Tsounis, M. Alge, J. Lee, F. Farshidian, and M. Hutter, "Deepgait: Planning and control of quadrupedal gaits using deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3699–3706, 2020.

[18] W. Yu, D. Jain, A. Escontrela, A. Iscen, P. Xu, E. Coumans, S. Ha, J. Tan, and T. Zhang, "Visual-locomotion: Learning to walk on complex terrains with vision," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Faust, D. Hsu, and G. Neumann, Eds., vol. 164. PMLR, 08–11 Nov 2022, pp. 1291–1302.

[19] H.-W. Park, P. M. Wensing, and S. Kim, "Jumping over obstacles with mit cheetah 2," *Robotics and Autonomous Systems*, vol. 136, p. 103703, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0921889020305431

[20] Q. Nguyen, M. J. Powell, B. Katz, J. D. Carlo, and S. Kim, "Optimized jumping on the mit cheetah 3 robot," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 7448–7454.

[21] S. H. Jeon, S. Kim, and D. Kim, "Real-time optimal landing control of the mit mini cheetah," 2021.

[22] N. Rudin, H. Kolvenbach, V. Tsounis, and M. Hutter, "Cat-like jumping and landing of legged robots in low gravity using deep reinforcement learning," *IEEE Transactions on Robotics*, 2021.

[23] V. Kurtz, H. Li, P. M. Wensing, and H. Lin, "Mini cheetah, the falling cat: A case study in machine learning and trajectory optimization for robot acrobatics," 2021.

[24] D. Hoeller, L. Wellhausen, F. Farshidian, and M. Hutter, "Learning a state representation and navigation in cluttered and dynamic environments," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5081–5088, 2021.

[25] C. Imai, M. Zhang, Y. Zhang, M. Kierebinski, R. Yang, Y. Qin, and X. Wang, "Vision-guided quadrupedal locomotion in the wild with multi-modal delay randomization," in *2022 IEEE/RSJ international conference on intelligent robots and systems (IROS)*.

[26] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Faust, D. Hsu, and G. Neumann, Eds., vol. 164. PMLR, 08–11 Nov 2022, pp. 91–100. [Online]. Available: https://proceedings.mlr.press/v164/rudin22a.html

[27] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac gym: High performance gpu-based physics simulation for robot learning," 2021.

[28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *CoRR*, vol. abs/1707.06347, 2017.

[29] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019. [Online]. Available: https://www.science.org/doi/abs/10.1126/scirobotics.aau5872

[30] N. Hemati and M. Leu, "A complete model characterization of brushless dc motors," *IEEE Transactions on Industry Applications*, vol. 28, no. 1, pp. 172–180, 1992.

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 02,2026 at 15:49:53 UTC from IEEE Xplore. Restrictions apply.